Quickstart

Sentence Transformer

Characteristics of Sentence Transformer (a.k.a bi-encoder) models:

1. Calculates a **fixed-size vector representation (embedding)** given **texts, images, audio, or video**.
2. Embedding calculation is often **efficient**, embedding similarity calculation is **very fast**.
3. Applicable for a **wide range of tasks**, such as semantic textual similarity, semantic search, clustering, classification, paraphrase mining, and more.
4. Often used as a **first step in a two-step retrieval process**, where a Cross-Encoder (a.k.a. reranker) model is used to re-rank the top-k results from the bi-encoder.

Once you have installed Sentence Transformers, you can easily use Sentence Transformer models:

Text | Multimodal

Documentation

1. `SentenceTransformer`
2. `SentenceTransformer.encode`
3. `SentenceTransformer.encode_query`
4. `SentenceTransformer.encode_document`
5. `SentenceTransformer.similarity`

Other useful methods and links:

- `SentenceTransformer.similarity_pairwise`
- SentenceTransformer > Usage
- SentenceTransformer > Speeding up Inference
- SentenceTransformer > Pretrained Models
- SentenceTransformer > Training Overview

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("Qwen/Qwen3-Embedding-0.6B")

query = "Which planet is known as the Red Planet?"
documents = [
    "Jupiter is the largest planet in our solar system.",
    "Mars gets its reddish color from iron minerals in its soil.",
    "Photosynthesis lets plants turn sunlight into energy.",
]

query_embedding = model.encode_query(query)
document_embeddings = model.encode_document(documents)
print(query_embedding.shape, document_embeddings.shape)
# (1024,) (3, 1024)

scores = model.similarity(query_embedding, document_embeddings)[0]
for index in scores.argsort(descending=True):
    print(f"{scores[index]:.4f}: {documents[index]}")
# 0.6115: Mars gets its reddish color from iron minerals in its soil.
# 0.4360: Jupiter is the largest planet in our solar system.
# 0.1893: Photosynthesis lets plants turn sunlight into energy.
```

- SentenceTransformer > Loss Overview
- SentenceTransformer > Training Examples

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

Finetuning Sentence Transformer models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Sentence Transformer > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Cross Encoder

Characteristics of Cross Encoder (a.k.a reranker) models:

1. Calculates a **similarity score** given **pairs of inputs** (typically text, but also images or other modalities).
2. Generally provides **superior performance** compared to a Sentence Transformer (a.k.a. bi-encoder) model.

3. Often **slower** than a Sentence Transformer model, as it requires computation for each pair rather than each text.
4. Due to the previous 2 characteristics, Cross Encoders are often used to **re-rank the top-k results** from a Sentence Transformer model.

The usage for Cross Encoder (a.k.a. reranker) models is similar to Sentence Transformers:

Text | Multimodal

Documentation

1. `CrossEncoder`
2. `CrossEncoder.rank`
3. `CrossEncoder.predict`

Other useful methods and links:

- CrossEncoder > Usage
- CrossEncoder > Speeding up Inference
- CrossEncoder > Pretrained Models
- CrossEncoder > Training Overview
- CrossEncoder > Loss Overview
- CrossEncoder > Training Examples

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

```python
from sentence_transformers import CrossEncoder

# 1. Load a pretrained CrossEncoder model
model = CrossEncoder("cross-encoder/stsb-distilroberta-base")

# We want to compute the similarity between the query sentence...
query = "A man is eating pasta."

# ... and all sentences in the corpus
corpus = [
    "A man is eating food.",
    "A man is eating a piece of bread.",
    "The girl is carrying a baby.",
    "A man is riding a horse.",
    "A woman is playing violin.",
    "Two men pushed carts through the woods.",
    "A man is riding a white horse on an enclosed ground.",
    "A monkey is playing drums.",
    "A cheetah is running behind its prey.",
]

# 2. We rank all sentences in the corpus for the query
ranks = model.rank(query, corpus)

# Print the scores
print("Query: ", query)
for rank in ranks:
    print(f"{rank['score']:.2f}\t{corpus[rank['corpus_id']]}")
"""
Query:  A man is eating pasta.
0.67    A man is eating food.
0.34    A man is eating a piece of bread.
0.08    A man is riding a horse.
0.07    A man is riding a white horse on an enclosed ground.
0.01    The girl is carrying a baby.
0.01    Two men pushed carts through the woods.
0.01    A monkey is playing drums.
0.01    A woman is playing violin.
0.01    A cheetah is running behind its prey.
"""

# 3. Alternatively, you can also manually compute the score between two sentences
import numpy as np

sentence_combinations = [[query, sentence] for sentence in corpus]
scores = model.predict(sentence_combinations)

# Sort the scores in decreasing order to get the corpus indices
ranked_indices = np.argsort(scores)[::-1]
print("Scores:", scores)
```

```
print("Indices:", ranked_indices)
"""
Scores: [0.6732372, 0.34102544, 0.00542465,
0.07569341, 0.00525378, 0.00536814, 0.06676237,
0.00534825, 0.00516717]
Indices: [0 1 3 6 2 5 7 4 8]
"""
```

With `CrossEncoder("cross-encoder/stsb-distilroberta-base")` we pick which CrossEncoder model we load. CrossEncoder models can also work with multimodal inputs: Qwen/Qwen3-VL-Reranker-2B can rank images and text by relevance to a query.

Finetuning CrossEncoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read CrossEncoder > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Sparse Encoder

Characteristics of Sparse Encoder models:

1. Calculates **sparse vector representations** where most dimensions are zero.
2. Provides **efficiency benefits** for large-scale retrieval systems due to the sparse nature of embeddings.
3. Often **more interpretable** than dense embeddings, with non-zero dimensions corresponding to specific tokens.
4. **Complementary to dense embeddings**, enabling hybrid search systems that combine the strengths of both approaches.

The usage for Sparse Encoder models follows a similar pattern to Sentence Transformers:

Documentation

1. `SparseEncoder`
2. `SparseEncoder.encode`
3. `SparseEncoder.similarity`
4. `SparseEncoder.sparsity`

```python
from sentence_transformers import SparseEncoder

# 1. Load a pretrained SparseEncoder model
model = SparseEncoder("naver/splade-cocondenser-ensembledistil")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate sparse embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 30522] - sparse representation with vocabulary size dimensions

# 3. Calculate the embedding similarities (using dot product by default)
similarities = model.similarity(embeddings, embeddings)
print(similarities)
# tensor([[   35.629,     9.154,     0.098],
#         [    9.154,    27.478,     0.019],
#         [    0.098,     0.019,    29.553]])

# 4. Check sparsity statistics
stats = SparseEncoder.sparsity(embeddings)
print(f"Sparsity: {stats['sparsity_ratio']:.2%}")  # Typically >99% zeros
print(f"Avg non-zero dimensions per embedding: {stats['active_dims']:.2f}")
```

Other useful methods and links:

- SparseEncoder > Usage
- SparseEncoder > Speeding up Inference
- SparseEncoder > Pretrained Models
- SparseEncoder > Training Overview
- SparseEncoder > Loss Overview
- Sparse Encoder > Vector Database Integration

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

With `SparseEncoder("naver/splade-cocondenser-ensembledistil")` we load a pretrained SPLADE model that generates sparse embeddings. SPLADE (SParse Lexical AnD Expansion) models use MLM prediction mechanisms to create sparse representations that are particularly effective for information retrieval tasks.

Finetuning Sparse Encoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Sparse Encoder > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Multi-Vector Encoder

Characteristics of Multi-Vector Encoder (a.k.a ColBERT-style or “late-interaction”) models:

1. Calculates a **sequence of token-level vectors** per input rather than a single fixed-size embedding.
2. Scores queries against documents with the **MaxSim** operator: for each query token, take the maximum similarity to any document token, then sum across query tokens.
3. Preserves **token-level matching information** that single-vector models discard, typically yielding stronger retrieval at the cost of a larger index footprint.
4. Recent VLM-backed variants (ColPali, ColQwen2, ColModernVBert, …) extend this to **image documents** (each image patch becomes a “token”), enabling end-to-end OCR-free document retrieval.

The usage for Multi-Vector Encoder models follows a similar pattern to Sentence Transformers:

Text | Multimodal

```python
from sentence_transformers import MultiVectorEncoder

# 1. Load a pretrained MultiVectorEncoder model
model = MultiVectorEncoder("lightonai/LateOn")

queries = ["What is the capital of France?"]
documents = [
    "Paris is the capital of France.",
    "Berlin is the capital of Germany.",
]

# 2. Encode queries and documents (note the
asymmetric encode_query / encode_document split)
query_embeddings = model.encode_query(queries)
document_embeddings =
model.encode_document(documents)

# Each entry is a 2D tensor of shape (num_tokens_i,
embedding_dim), variable-length per input.
print(query_embeddings[0].shape)
# torch.Size([10, 128])

# 3. Score with MaxSim
scores = model.similarity(query_embeddings,
document_embeddings)
print(scores)
# tensor([[9.1129, 8.8769]], device='cuda:0')
```

Documentation

1. `MultiVectorEncoder`
2. `MultiVectorEncoder.encode_query`
3. `MultiVectorEncoder.encode_document`
4. `MultiVectorEncoder.similarity`

Other useful methods and links:

- MultiVectorEncoder > Usage
- MultiVectorEncoder > Speeding up Inference
- MultiVectorEncoder > Pretrained Models
- MultiVectorEncoder > Training Overview
- MultiVectorEncoder > Loss Overview

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access

With `MultiVectorEncoder("lightonai/LateOn")` we pick a pretrained late-interaction model. Late-interaction models are particularly strong for **retrieval** tasks where token-level matching matters (named entities, exact-phrase queries, sub-document relevance). ColPali-style variants extend this to image document retrieval over scanned pages, slides, and figures.

to private or gated models your account can read. See the login instructions.

Finetuning Multi-Vector Encoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Multi-Vector Encoder > Speeding up Inference for benchmarks and tips on how to speed up inference with the ONNX and OpenVINO backends.

Next Steps

Consider reading one of the following sections next:

- Sentence Transformers > Usage
- Sentence Transformers > Pretrained Models
- Sentence Transformers > Training Overview
- Sentence Transformers > Training Examples > Multilingual Models
- Sentence Transformers > Training Examples > Multimodal Models
- Cross Encoder > Usage
- Cross Encoder > Pretrained Models
- Cross Encoder > Training Examples > Multimodal Models
- Sparse Encoder > Usage
- Sparse Encoder > Pretrained Models
- Sparse Encoder > Vector Database Integration
- Multi-Vector Encoder > Usage
- Multi-Vector Encoder > Pretrained Models
- Multi-Vector Encoder > Training Overview